

DSRAC SIS: Vectorization

3. assignment

Luka Šveigl, ls6727@student.uni-lj.si, 63200301

I. INTRODUCTION

As part of the third assignment of our Computer systems course, we were required to vectorize the K-means clustering algorithm using AVX/AV2 intrinsics. The K-means clustering algorithm is an unsupervised machine-learning technique used for clustering data based on similarities in their features. In this report we describe our double, float and unsigned 8 bit vectorized implementations, before presenting the runtimes in terms of cycles and seconds. We conclude the report with a discussion and interpretation of the results.

II. METHODOLOGY

A. Double image implementation

First, we were required to vectorize the K-means clustering algorithm operating on an image and the cluster centroids represented as double-precision floating point numbers. To efficiently vectorize the algorithm, we pack a 256-bit vector with 4 64-bit centroids, meaning we can efficiently process 4 clusters at a time. Our *assign_points_to_clusters* implementation focuses on vectorizing the algorithm in the direction of clusters, meaning we compute the distances from one pixels to multiple clusters. An alternative approach would be vectorization in the direction of pixels, where we would compute the distances from multiple pixels to one cluster.

To vectorize the *update_centroids* function, we simply employed partial sums and processed 4 pixels at a time for each cluster.

B. Float image implementation

Our next implementation was the vectorized implementation of the algorithm operating on an image and cluster centroids represented as single-precision floating point numbers. The *assign_points_to_clusters* implementation once again focuses on vectorizing the algorithm in the direction of clusters. To efficiently vectorize the algorithm, we pack a 256-bit vector with 8 32-bit centroids, meaning we can process 8 clusters at a time. This approach works well, however maximum performance is not achieved when executing the algorithm with 4 clusters. Vectorizing the algorithm in the direction of pixels would be a more suitable approach here.

Similar to the double implementation, to vectorize the *update_centroids* function, we simply employed partial sums and processed 8 pixels at a time for each cluster.

C. Uint8 image implementation

Our final implementation was the vectorized implementation of the algorithm operating on an image and cluster centroids represented as unsigned 8-bit integers. The *assign_points_to_clusters* implementation focuses on vectorizing the algorithm in the direction of clusters, where we process 8 pixels at a time. The reason for this decision the fact that we cast the pixels into single-precision floating point numbers to compute the distances in as much precision as possible. In retrospect, this is not necessary, and we could have processed 16 pixels at a time with no loss of precision.

In the unsigned 8-bit integer implementation, we once again employed partial sums and processed 8 pixels at a time. In retrospect, we could have processed 16 pixels at a time and achieved even bigger speed-ups.

III. RESULTS

We evaluated our double-precision floating point, single-precision floating point and unsigned 8-bit integer implementation on algorithm versions with 4, 8 and 16 clusters, measuring the run-times in terms of cycles and seconds. We executed each variant of the algorithm 10 times and averaged the results in order to reduce the impact of outlier results caused by the random initial centroid distribution. The execution times of the scalar implementation are presented in Table I, while the results of the double-precision floating point are presented in Table II, the results of single-precision floating point results are presented in Table III and the unsigned 8-bit integer results are presented in Table IV. The speed-ups of these implementations are presented in Table V.

Clusters	Cycles	Seconds
4	24821085868	12.410543
8	18221794598	9.1108972
16	34242265376	17.1211328

TABLE I: The average run-times of the scalar implementation.

Clusters	Cycles	Seconds
4	28778775984	14.3893881
8	17748634668	8.8743175
16	26227791194	13.1138955

TABLE II: The average run-times of the double implementation.

Clusters	Cycles	Seconds
4	25822562974	12.9112814
8	15420883124	7.7104415
16	21400426502	10.7002133

TABLE III: The average run-times of the float implementation.

Clusters	Cycles	Seconds
4	6334646720	3.1673234
8	2487745646	1.2438728
16	5157865246	2.5789327

TABLE IV: The average run-times of the unsigned integer implementation.

Clusters	t_s/t_{dp}	t_s/t_{sp}	t_s/t_{uint}
4	0.8624788569	0.9612169866	3.918306227
8	1.026658918	1.181631065	7.324621296
16	1.305571849	1.600073972	6.638844356

TABLE V: The speed-ups of the various implementations.

IV. DISCUSSION

In this chapter, we delve into the results obtained from the vectorized implementations of the K-Means clustering algorithm using AVX/AV2 intrinsics. The performance is evaluated based on execution time and speed-up across different data types and cluster sizes.

The performance comparison reveals interesting insights. The scalar implementation, lacking vectorization, exhibits the highest execution times among all variants. This is expected since scalar operations cannot exploit the benefits of vectorization for efficient computation, particularly evident in large-scale computations.

Comparatively, the double precision implementation shows improved performance over the scalar version across with cluster sizes of 8 and 16, showing worse performance in the case of 4 clusters. Double precision arithmetic tends to be more computationally intensive, benefiting more from vectorized processing. However, the gains are not as substantial as observed with other data types.

The float implementation performs better than the double precision one, indicating that the reduction in precision allows for more efficient vectorization and faster execution times due to the fact the algorithm can process

more elements at once. This highlights the importance of data type selection in optimizing performance for specific applications.

The unsigned integer implementation emerges as the best performer in terms of runtime among all tested variants. This suggests that for this particular application, utilizing unsigned integer arithmetic with vectorization significantly outperforms both double and float implementations. The simplicity of integer operations and potentially more efficient use of vectorized instructions could explain this observation.

The speed-up analysis further supports these findings. Comparing each vectorized implementation to the scalar version, we observe varying degrees of speed-up. The double precision implementation achieves moderate to significant improvements, while the float implementation demonstrates the highest speed-up among all comparisons. However, the unsigned integer implementation exhibits the most substantial speed-up, indicating its superior performance for this specific scenario.

In conclusion, the vectorized implementations of the K-Means clustering algorithm using AVX/AV2 intrinsics demonstrate significant performance improvements over the scalar implementation across all tested data types. The choice of data type significantly influences the speed-up achieved, with the unsigned integer implementation outperforming the others in this specific scenario. However, it's crucial to weigh the trade-offs between precision, range, and performance when selecting the appropriate data type for a given application.